

ChatGPT Prompts and replies for SODA

GPT-4o

Prompt 1 (No references given):

```
// -----// IR Dump After ExpandStridedMetadata (expand-strided-metadata) //-
----- //

#map = affine_map<(d0, d1) -> (d0 + d1)>

module attributes {soda.bambu.container_module, soda.container_module} {

  func.func @main_kernel(%arg0: memref<1x14x14x128xf32> {llvm.noalias},
    %arg1: memref<3x3x128x1xf32> {llvm.noalias}, %arg2:
    memref<1x12x12x1xf32> {llvm.noalias}) {

    cf.br ^bb1

  ^bb1: // pred: ^bb0

    affine.for %arg3 = 0 to 1 {

      affine.for %arg4 = 0 to 12 {

        affine.for %arg5 = 0 to 12 {

          affine.for %arg6 = 0 to 1 {

            affine.for %arg7 = 0 to 3 {

              affine.for %arg8 = 0 to 3 {

                affine.for %arg9 = 0 to 128 {

                  %0 = affine.apply #map(%arg4, %arg7)

                  %1 = affine.apply #map(%arg5, %arg8)

                  %2 = affine.load %arg0[%arg3, %0, %1, %arg9]:
                memref<1x14x14x128xf32>
```

```

        %3 = affine.load %arg1[%arg7, %arg8, %arg9, %arg6] :
memref<3x3x128x1xf32>

        %4 = affine.load %arg2[%arg3, %arg4, %arg5, %arg6] :
memref<1x12x12x1xf32>

        %5 = arith.mulf %2, %3 : f32

        %6 = arith.addf %4, %5 : f32

        affine.store %6, %arg2[%arg3, %arg4, %arg5, %arg6] :
memref<1x12x12x1xf32>

    }

}

}

}

}

}

}

return

}

```

} Optimize this code with loop tiling for MLIR-HLS + ASIC Design

Result:

```

#map = affine_map<(d0, d1) -> (d0 + d1)>

module attributes {soda.bambu.container_module, soda.container_module}{

  func.func @main_kernel(%arg0: memref<1x14x14x128xf32> {llvm.noalias},
        %arg1: memref<3x3x128x1xf32> {llvm.noalias},

```

```

    %arg2: memref<1x12x12x1xf32> {llvm.noalias}) {
cf.br ^bb1
^bb1:
  affine.for %arg3 = 0 to 1 {
    // Tile loops over output height (arg4) and width (arg5)
    affine.for %t4 = 0 to 12 step 4 {
      affine.for %t5 = 0 to 12 step 4 {
        affine.for %arg4 = %t4 to affine.min(%t4 + 4, 12) {
          affine.for %arg5 = %t5 to affine.min(%t5 + 4, 12) {
            affine.for %arg6 = 0 to 1 {
              affine.for %arg7 = 0 to 3 {
                affine.for %arg8 = 0 to 3 {
                  // Tile channel loop (arg9)
                  affine.for %t9 = 0 to 128 step 32 {
                    affine.for %arg9 = %t9 to affine.min(%t9 + 32, 128) {
                      %0 = affine.apply #map(%arg4, %arg7)
                      %1 = affine.apply #map(%arg5, %arg8)
                      %2 = affine.load %arg0[%arg3, %0, %1, %arg9] :
memref<1x14x14x128xf32>
                      %3 = affine.load %arg1[%arg7, %arg8, %arg9, %arg6] :
memref<3x3x128x1xf32>
                      %4 = affine.load %arg2[%arg3, %arg4, %arg5, %arg6] :
memref<1x12x12x1xf32>
                      %5 = arith.mulf %2, %3 : f32

```

```

        %6 = arith.addf %4, %5 : f32

        affine.store %6, %arg2[%arg3, %arg4, %arg5, %arg6] :
memref<1x12x12x1xf32>

    }

}

}

}

}

}

}

}

}

}

return

}

}

```

Outcome: Doesn't work due to syntax error.

output_llm/04bAlexNet_10_conv2d_3_tile__trial_1.mlir:22:69: error: expected
'\)' in affine map range

```

%cond4 = affine.apply affine_map<(d0) -> (d0 < 12)>(%i4)

```

^

Prompt 2 (with reference):

```
// -----// IR Dump After AffineLoopTiling (affine-loop-tile) //----- //  
  
func.func @main_kernel(%arg0: memref<1x14x14x128xf32> {llvm.noalias},  
%arg1: memref<3x3x128x1xf32> {llvm.noalias}, %arg2:  
memref<1x12x12x1xf32> {llvm.noalias}) {  
  
  cf.br ^bb1  
  
^bb1: // pred: ^bb0  
  
  affine.for %arg3 = 0 to 1 {  
  
    affine.for %arg4 = 0 to 12 step 12 {  
  
      affine.for %arg5 = 0 to 12 step 12 {  
  
        affine.for %arg6 = 0 to 1 {  
  
          affine.for %arg7 = 0 to 3 step 3 {  
  
            affine.for %arg8 = 0 to 3 step 3 {  
  
              affine.for %arg9 = 0 to 128 step 8 {  
  
                affine.for %arg10 = affine_map<(d0) -> (d0)>(%arg3) to  
affine_map<(d0) -> (d0 + 1)>(%arg3) {  
  
                  affine.for %arg11 = affine_map<(d0) -> (d0)>(%arg4) to  
affine_map<(d0) -> (d0 + 12)>(%arg4) {  
  
                    affine.for %arg12 = affine_map<(d0) -> (d0)>(%arg5) to  
affine_map<(d0) -> (d0 + 12)>(%arg5) {  
  
                      affine.for %arg13 = affine_map<(d0) -> (d0)>(%arg6) to  
affine_map<(d0) -> (d0 + 1)>(%arg6) {  
  
                        affine.for %arg14 = affine_map<(d0) -> (d0)>(%arg7) to  
affine_map<(d0) -> (d0 + 3)>(%arg7) {
```

```

    affine.for %arg15 = affine_map<(d0) -> (d0)>(%arg8) to
affine_map<(d0) -> (d0 + 3)>(%arg8) {

        affine.for %arg16 = affine_map<(d0) -> (d0)>(%arg9) to
affine_map<(d0) -> (d0 + 8)>(%arg9) {

            %0 = affine.apply affine_map<(d0, d1) -> (d0 + d1)>(%arg11,
%arg14)

            %1 = affine.apply affine_map<(d0, d1) -> (d0 + d1)>(%arg12,
%arg15)

            %2 = affine.load %arg0[%arg10, %0, %1, %arg16] :
memref<1x14x14x128xf32>

            %3 = affine.load %arg1[%arg14, %arg15, %arg16, %arg13] :
memref<3x3x128x1xf32>

            %4 = affine.load %arg2[%arg10, %arg11, %arg12, %arg13] :
memref<1x12x12x1xf32>

            %5 = arith.mulf %2, %3 : f32

            %6 = arith.addf %4, %5 : f32

            affine.store %6, %arg2[%arg10, %arg11, %arg12, %arg13] :
memref<1x12x12x1xf32>

        }

    }

}

}

}

}

}

}

}

```

```

    }
  }
}
}
}
}
return
}

```

With the above reference for post tiling syntax, optimize the below code using loop tiling for MLIR-HLS + ASIC design

```

// -----// IR Dump After ExpandStridedMetadata (expand-strided-metadata) //-
---- //

#map = affine_map<(d0, d1) -> (d0 + d1)>

module attributes {soda.bambu.container_module, soda.container_module} {
  func.func @main_kernel(%arg0: memref<1x14x14x128xf32> {llvm.noalias},
    %arg1: memref<3x3x128x1xf32> {llvm.noalias}, %arg2:
    memref<1x12x12x1xf32> {llvm.noalias}) {
    cf.br ^bb1
  ^bb1: // pred: ^bb0
    affine.for %arg3 = 0 to 1 {
      affine.for %arg4 = 0 to 12 {
        affine.for %arg5 = 0 to 12 {

```

```

    affine.for %arg6 = 0 to 1 {
      affine.for %arg7 = 0 to 3 {
        affine.for %arg8 = 0 to 3 {
          affine.for %arg9 = 0 to 128 {
            %0 = affine.apply #map(%arg4, %arg7)
            %1 = affine.apply #map(%arg5, %arg8)
            %2 = affine.load %arg0[%arg3, %0, %1, %arg9] :
memref<1x14x14x128xf32>

            %3 = affine.load %arg1[%arg7, %arg8, %arg9, %arg6] :
memref<3x3x128x1xf32>

            %4 = affine.load %arg2[%arg3, %arg4, %arg5, %arg6] :
memref<1x12x12x1xf32>

            %5 = arith.mulf %2, %3 : f32
            %6 = arith.addf %4, %5 : f32

            affine.store %6, %arg2[%arg3, %arg4, %arg5, %arg6] :
memref<1x12x12x1xf32>
          }
        }
      }
    }
  }
}
}
return

```

```
}  
}
```

Output:

```
#map = affine_map<(d0, d1) -> (d0 + d1)>  
module attributes {soda.bambu.container_module, soda.container_module}{  
  func.func @main_kernel(  
    %arg0: memref<1x14x14x128xf32> {llvm.noalias},  
    %arg1: memref<3x3x128x1xf32> {llvm.noalias},  
    %arg2: memref<1x12x12x1xf32> {llvm.noalias}  
  ){  
    cf.br ^bb1  
^bb1: // pred: ^bb0  
    affine.for %arg3 = 0 to 1 {  
      affine.for %arg4 = 0 to 12 step 4 {  
        affine.for %arg5 = 0 to 12 step 4 {  
          affine.for %arg6 = 0 to 1 {  
            affine.for %arg7 = 0 to 3 step 3 {  
              affine.for %arg8 = 0 to 3 step 3 {  
                affine.for %arg9 = 0 to 128 step 8 {  
                  affine.for %i4 = affine_map<(d0) -> (d0)>(%arg4) to affine_map<(d0) -  
> (d0 + 4)>(%arg4) {  
                    affine.for %i5 = affine_map<(d0) -> (d0)>(%arg5) to affine_map<(d0) -  
> (d0 + 4)>(%arg5) {
```

```

    affine.for %i7 = affine_map<(d0) -> (d0)>(%arg7) to affine_map<(d0)
-> (d0 + 3)>(%arg7) {

        affine.for %i8 = affine_map<(d0) -> (d0)>(%arg8) to
affine_map<(d0) -> (d0 + 3)>(%arg8) {

            affine.for %i9 = affine_map<(d0) -> (d0)>(%arg9) to
affine_map<(d0) -> (d0 + 8)>(%arg9) {

                %0 = affine.apply #map(%i4, %i7)

                %1 = affine.apply #map(%i5, %i8)

                %2 = affine.load %arg0[%arg3, %0, %1, %i9] :
memref<1x14x14x128xf32>

                %3 = affine.load %arg1[%i7, %i8, %i9, %arg6] :
memref<3x3x128x1xf32>

                %4 = affine.load %arg2[%arg3, %i4, %i5, %arg6] :
memref<1x12x12x1xf32>

                %5 = arith.mulf %2, %3 : f32

                %6 = arith.addf %4, %5 : f32

                affine.store %6, %arg2[%arg3, %i4, %i5, %arg6] :
memref<1x12x12x1xf32>

            }

        }

    }

}

```

```

    }
  }
}
}
}
return
}
}

```

The code works for MLIR-HLS. Need to perform Design Space Exploration utilizing this prompt.

Prompt 3 (with toy MLIR file):

```

// -----// IR Dump After ExpandStridedMetadata (expand-strided-metadata) //-
---- //

#map = affine_map<(d0, d1) -> (d0 + d1)>

module attributes {soda.bambu.container_module, soda.container_module} {
  func.func @main_kernel(%arg0: memref<1x14x14x128xf32> {llvm.noalias},
    %arg1: memref<3x3x128x1xf32> {llvm.noalias}, %arg2:
    memref<1x12x12x1xf32> {llvm.noalias}) {
    cf.br ^bb1
  ^bb1: // pred: ^bb0
    affine.for %arg3 = 0 to 1 {
      affine.for %arg4 = 0 to 12 {
        affine.for %arg5 = 0 to 12 {

```

```

    affine.for %arg6 = 0 to 1 {
      affine.for %arg7 = 0 to 3 {
        affine.for %arg8 = 0 to 3 {
          affine.for %arg9 = 0 to 128 {
            %0 = affine.apply #map(%arg4, %arg7)
            %1 = affine.apply #map(%arg5, %arg8)
            %2 = affine.load %arg0[%arg3, %0, %1, %arg9] :
memref<1x14x14x128xf32>

            %3 = affine.load %arg1[%arg7, %arg8, %arg9, %arg6] :
memref<3x3x128x1xf32>

            %4 = affine.load %arg2[%arg3, %arg4, %arg5, %arg6] :
memref<1x12x12x1xf32>

            %5 = arith.mulf %2, %3 : f32
            %6 = arith.addf %4, %5 : f32

            affine.store %6, %arg2[%arg3, %arg4, %arg5, %arg6] :
memref<1x12x12x1xf32>

          }
        }
      }
    }
  }
}
}
return

```

}

} Optimize this code with loop tiling for MLIR-HLS + ASIC Design with toy MLIR reference.

Output: Syntax error. It does not understand how to apply loop tiling from the Toy MLIR file.